

Projeto de App de Compras

Arquitetura técnica, modelo de dados e SQL inicial

Documento consolidando a evolução da conversa: proposta de um aplicativo mobile para lista de compras, captura de nota fiscal, construção de base colaborativa de preços e recomendação do mercado mais barato com base na lista do usuário.

Objetivo do produto. Transformar uma funcionalidade aparentemente simples — lista de compras — em um motor de coleta de dados e inteligência de preços por mercado, com potencial para histórico de inflação local, alertas e recomendação personalizada.

Estratégia recomendada. Começar pequeno: MVP focado em lista, captura de nota via QR Code/NFC-e, extração de itens, normalização de produtos e comparação determinística entre mercados.

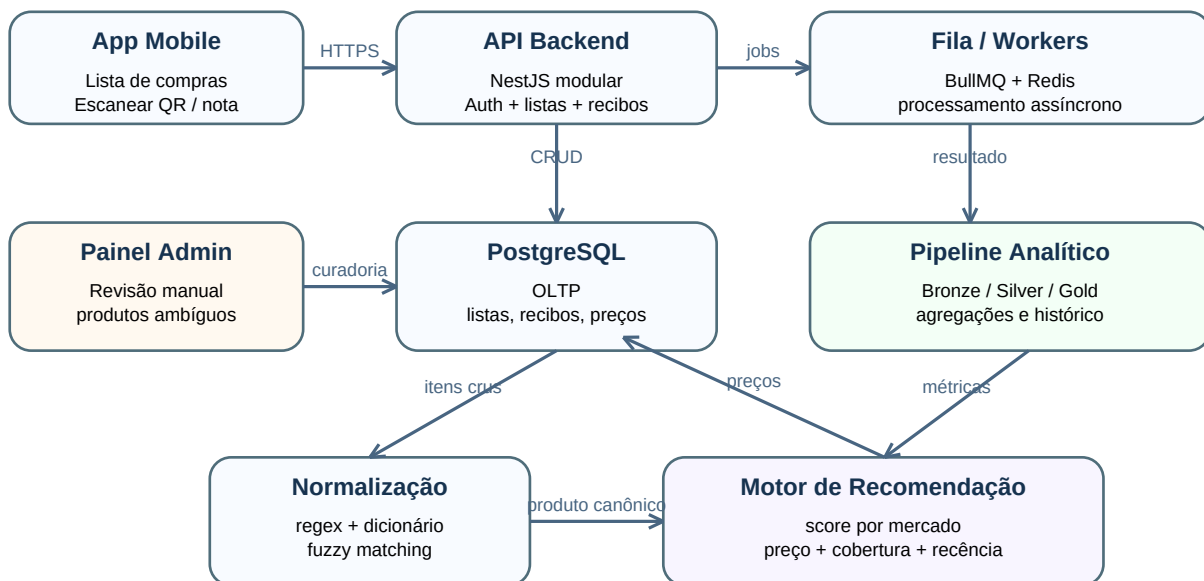
Stack sugerida. React Native + NestJS + PostgreSQL + Redis/BullMQ + painel admin web.

Bloco	Papel no sistema
App mobile	Captura da intenção do usuário: lista, escaneamento e consulta de recomendação.
Backend	Expõe APIs, autentica, gerencia listas e orquestra processamento.
Workers	Processam nota fiscal, normalizam produtos e atualizam observações de preço.
Banco transacional	Guarda usuários, listas, recibos, itens e preços observados.
Camada analítica	Consolida métricas de preço, cobertura, recência e histórico por mercado.

1. Arquitetura lógica do sistema

A solução recomendada para a sua realidade não é um ecossistema de microserviços. É um **monólito modular** com processamento assíncrono, porque isso reduz complexidade operacional sem matar a capacidade de crescimento.

Diagrama lógico da solução



Arquitetura recomendada para MVP: monólito modular + fila assíncrona + camada analítica leve.

Decisão central. O backend atende o app em tempo real, mas o processamento pesado da nota fiscal não deve acontecer dentro da requisição principal. O fluxo ideal é: receber QR/nota → persistir como pendente → publicar job → processar em worker → devolver resultado.

Módulo	Responsabilidade	Observação prática
Auth	Cadastro, login, refresh token, controle de acesso	JWT + refresh token já resolve o MVP.
Shopping Lists	Listas, itens, histórico e favoritos	É o ponto de entrada da jornada do usuário.
Receipts	Recebimento de QR, payload cru e status do processamento	Salvar o dado bruto evita retrabalho.
Products	Catálogo canônico e regras de normalização	É o coração do valor analítico.
Pricing	Observações de preço e agregações	Base para comparação entre mercados.
Recommendations	Cálculo de score e ranking por mercado	Começar determinístico, sem IA mística.
Admin	Revisão manual de produtos ambíguos e erros	Sem backoffice, a operação trava.

2. Modelo conceitual do banco de dados

O banco transacional precisa refletir duas coisas ao mesmo tempo: **operação do app** e **rastreabilidade dos dados de preço**. O desenho abaixo prioriza consistência, auditabilidade e facilidade de evolução.

Tabela	Finalidade	Chaves principais
users	Conta do usuário e preferências básicas	id, email
markets	Mercados e filiais com localização	id, city, latitude, longitude
shopping_lists	Cabeçalho da lista do usuário	id, user_id
shopping_list_items	Itens de cada lista	id, list_id, canonical_product_id
receipts	Recibo/nota fiscal recebida	id, user_id, market_id, access_key
receipt_items	Itens extraídos da nota	id, receipt_id, canonical_product_id
canonical_products	Produto canônico padronizado	id, slug
product_aliases	Descrições alternativas observadas	id, canonical_product_id, alias_text
price_observations	Observações de preço por mercado e data	id, canonical_product_id, market_id, observed_at
market_product_snapshots	Tabela agregada para consulta rápida	market_id, canonical_product_id, snapshot_date

Relações mais importantes. Um usuário possui várias listas. Uma nota fiscal pertence a um usuário e a um mercado. Uma nota possui vários itens. Cada item pode ser associado a um produto canônico. Cada item válido gera uma observação de preço. Produtos canônicos podem ter vários aliases capturados da vida real.

Tabela	Colunas essenciais (resumo)
users	id, full_name, email, password_hash, city, created_at
markets	id, name, chain_name, city, neighborhood, latitude, longitude, created_at
shopping_lists	id, user_id, name, status, created_at, updated_at
shopping_list_items	id, list_id, raw_text, canonical_product_id, quantity, unit, checked
receipts	id, user_id, market_id, source_type, access_key, raw_payload, purchase_date, total_amount, status
receipt_items	id, receipt_id, raw_description, canonical_product_id, quantity, unit, unit_price, total_price, confidence_score
canonical_products	id, slug, canonical_name, category, brand, package_size, package_unit, attributes_json
price_observations	id, canonical_product_id, market_id, observed_price, observed_at, receipt_item_id

3. Regras de processamento e recomendação

O maior erro seria imaginar que o diferencial está no app. O diferencial está na **qualidade da normalização** e na **confiança do preço observado**. Por isso, a arquitetura de dados precisa assumir incerteza desde o começo.

Etapa	O que acontece	Risco real	Tratamento recomendado
Ingestão	QR/nota é recebido e salvo cru	Perda de informação	Persistir payload original sempre.
Extração	Itens e metadados são extraídos	Diferença entre estados/fontes	Criar adaptadores por fonte.
Normalização	Descrição vira produto canônico	Texto inconsistente	Regex + dicionário + fuzzy matching.
Validação	Checa duplicidade e plausibilidade	Fraude ou nota repetida	Hash, chave de acesso, regras de unicidade.
Observação	Preço válido é gravado	Preço muito antigo	Persistir recência e origem.
Recomendação	Mercados são ranqueados	Lista incompleta ou preços faltantes	Aplicar penalidade por item ausente.

Fórmula inicial de score. $\text{score} = \text{preço_estimado} + \text{penalidade_por_item_ausente} + \text{penalidade_por_preço_antigo}$. Isso é simples, explicável e suficiente para o MVP.

Camada analítica recomendada. Bronze para payload bruto; Silver para itens limpos e vinculados; Gold para preço médio por mercado, variação semanal, cobertura por lista e histórico local.

4. SQL inicial para PostgreSQL

Abaixo está o conjunto de tabelas-base recomendado para um primeiro ciclo de desenvolvimento. O arquivo SQL completo acompanha este PDF em separado.

Objeto	Propósito
users	Cadastro de usuários
markets	Mercados e filiais
canonical_products	Catálogo canônico
product_aliases	Aliases observados nas notas
shopping_lists	Cabeçalho das listas
shopping_list_items	Itens das listas
receipts	Notas/recibos recebidos
receipt_items	Itens extraídos das notas
price_observations	Preços observados válidos
market_product_snapshots	Agregado diário para recomendação rápida
vw_market_latest_prices	View com preço mais recente por mercado/produto

Índices essenciais. email em users, localização/nomes de markets, slug de canonical_products, access_key em receipts, e índices compostos em price_observations para (canonical_product_id, market_id, observed_at desc).

Próximo passo técnico. Depois do schema, você deveria partir para: 1) API de listas, 2) ingestão de recibo, 3) worker de normalização, 4) query de recomendação.

Arquivos entregues neste pacote:

Arquivo	Conteúdo
projeto_lista_compras_arquitetura_bd.pdf	Documento consolidado com arquitetura, modelo conceitual e orientação de implementação.
projeto_lista_compras_modelo_inicial.sql	Schema inicial PostgreSQL com tabelas, constraints, índices e view base.